CS217 Assignment 6

Zhige Chen 12413315

Department of Computer Science and Engineering, Southern University of Science and Technology, China

October 18, 2025

Abstract

This project conducts a comprehensive performance evaluation comparing database management systems (PostgreSQL and openGauss) against manual file operations (including the JSON and BEVE format) using an employee database schema. This experiments reveal that PostgreSQL is generally faster than openGauss, and using DBMS can lead to less resource usage and provide significant advantages for complex queries and maintainability.

Contents

1 Project Overview

The **database management systems** (DBMS) have become fundamental to modern data management, but understanding their performance characteristics compared to traditional file operations remains crucial. This projects evaluates PostgreSQL and openGauss against file-based data management using a realistic employee database schema. The benchmarks are implemented in C++.

2 Experimental Setup

2.1 Environment Configuration

- **Hardware**: AMD Ryzen 9 7945HX, 64GB DDR4 4800MHz RAM, SK Hynix PC801 2T SSD
- **PostgreSQL**: Version 18.0.1, default configuration, running in Docker container
- **openGauss**: Version 7.0.0-RC1 build 10d38387, default configuration, running in Docker container
- **Docker**: Version 28.5.1, build e180ab8
- **C++ Compiler**: Clang 21.1.1, target x86_64-w64-windows-gnu
- **Dataset**: Employee data with 3919015 records and 333MB installed size, see the Github repository

2.2 Libraries Used

2.3 Measurement Methodology

3 Experiment Results

3.1 Data Retrieval Performance Comparison

3.2 Data Update Performance Comparison

3.3 Advanced Comparisons: Different File Format

4 Analysis and Discussion

5 Conclusion

A Appendix A: Source Code Repository

B Appendix B: Generative AI Usage

C Appendix C: Raw Experimental Data

C.1 Retrieval Test 1

Query	Samples	Iterations	Mean Time	Std Dev	Est. Run Time
LIMIT 100	10	1	665.128 µs	218.046 µs	4.650 05 ms
LIMIT 500	10	1	615.008 µs	211.992 µs	5.109 92 ms
LIMIT 1000	10	1	710.618 µs	153.451 µs	5.768 76 ms
LIMIT 5000	10	1	1.405 99 ms	582.315 µs	11.6519 ms
LIMIT 10000	10	1	2.487 25 ms	719.83 µs	19.8189 ms
LIMIT 50000	10	1	5.668 98 ms	927.226 µs	63.1476 ms
LIMIT 100000	10	1	9.828 08 ms	827.53 µs	105.203 ms
LIMIT 500000	10	1	46.985 ms	3.956 93 ms	469.665 ms
LIMIT 1000000	10	1	91.9645 ms	2.9558 ms	906.421 ms

Table 1: PostgreSQL SELECT Performance Benchmark Results

Query	Samples	Iterations	Mean Time	Std Dev	Est. Run Time
LIMIT 100	10	1	571.108 µs	179.701 µs	4.9526 ms
LIMIT 500	10	1	814.068 µs	346.285 µs	5.786 84 ms
LIMIT 1000	10	1	722.528 µs	238.369 µs	6.590 98 ms
LIMIT 5000	10	1	1.130 25 ms	243.238 µs	11.1561 ms
LIMIT 10000	10	1	1.782 46 ms	202.507 µs	17.5614 ms
LIMIT 50000	10	1	6.641 21 ms	690.447 µs	63.5856 ms
LIMIT 100000	10	1	12.6365 ms	968.095 µs	123.897 ms
LIMIT 500000	10	1	60.994 ms	2.7992 ms	592.659 ms
LIMIT 1000000	10	1	118.201 ms	2.235 25 ms	1.202 41 s

Table 2: openGauss SELECT Performance Benchmark Results

Benchmark Name	Samples	Iterations	Mean Time	Std Dev	Est. Run Time
File Loading	10	1	262.61 ms	14.152 ms	4.691 28 s
Reading 100	10	68	342.324 ns	79.9492 ns	226.44 µs
Reading 500	10	34	690.529 ns	179.585 ns	225.76 µs
Reading 1000	10	23	1.076 87 µs	307.625 ns	227.7 µs
Reading 5000	10	6	4.678 µs	1.762 52 µs	243.6 µs
Reading 10000	10	3	11.126 µs	4.811 44 µs	276.63 µs
Reading 50000	10	1	93.458 µs	89.8841 µs	415.4 µs
Reading 100000	10	1	283.348 µs	84.9363 µs	2.271 96 ms
Reading 500000	10	1	2.121 96 ms	434.837 µs	18.9072 ms
Reading 1000000	10	1	4.362 52 ms	291.284 µs	41.0047 ms

Table 3: JSON File Reading Performance Benchmark Results

D Appendix D: Test Code

D.1 Retrieval Test 1

```
#define SALARY_BENCH_PG(LIM)
↪  \
    BENCHMARK_ADVANCED("PostgreSQL salary ORDER BY employee_id LIMIT "
    ↪  #LIM)(Catch::Benchmark::Chronometer meter) {    \
        pqxx::connection conn{pg_conn_str.data()};
        ↪  \
        pqxx::read_transaction tx{conn};
        ↪  \
        tx.exec("SET search_path TO employees");
        ↪  \
        meter.measure([&] {
        ↪  \
            std::int64_t sum = 0;
            ↪  \
            for (auto [amount]:
            ↪  \
                 tx.query<std::int64_t>("SELECT amount FROM salary ORDER BY
                 ↪  employee_id LIMIT " #LIM)) {               \
                sum += amount;
                ↪  \
            }
            ↪  \
            return sum;
            ↪  \
        });
        ↪  \
    }
```

```
TEST_CASE("retrieval-test-1-postgresql",
↪  "[retrieval-post][retrieval-test-1][retrieval-test]") {
    try {
        SALARY_BENCH_PG(100);
        SALARY_BENCH_PG(500);
        SALARY_BENCH_PG(1000);
        SALARY_BENCH_PG(5000);
        SALARY_BENCH_PG(10000);
        SALARY_BENCH_PG(50000);
        SALARY_BENCH_PG(100000);
        SALARY_BENCH_PG(500000);
        SALARY_BENCH_PG(1000000);
    } catch (pqxx::sql_error const &e) {
        FAIL(std::format("Query failed with: {}", e.what()));
    } catch (std::exception const &e) {
        FAIL(std::format("Failed to connect to PostgreSQL server '{}'",
        ↪  e.what()));
    }
}
```

```
#define SALARY_BENCH_OG(LIM)
↪  \
    BENCHMARK_ADVANCED("openGauss salary ORDER BY employee_id LIMIT "
    ↪  #LIM)(Catch::Benchmark::Chronometer meter) {     \
        pqxx::connection conn{og_conn_str.data()};
        ↪  \
        pqxx::read_transaction tx{conn};
        ↪  \
        tx.exec("SET search_path TO employees");
        ↪  \
        meter.measure([&] {
        ↪  \
            std::int64_t sum = 0;
            ↪  \
            for (auto [amount]:
            ↪  \
                 tx.query<std::int64_t>("SELECT amount FROM salary ORDER BY
                 ↪  employee_id LIMIT " #LIM)) {               \
                sum += amount;
                ↪  \
            }
            ↪  \
            return sum;
            ↪  \
        });
        ↪  \
    }

TEST_CASE("retrieval-test-1-opengauss",
↪  "[retrieval-open][retrieval-test-1][retrieval-test]") {
    try {
        SALARY_BENCH_OG(100);
        SALARY_BENCH_OG(500);
        SALARY_BENCH_OG(1000);
        SALARY_BENCH_OG(5000);
        SALARY_BENCH_OG(10000);
        SALARY_BENCH_OG(50000);
        SALARY_BENCH_OG(100000);
        SALARY_BENCH_OG(500000);
        SALARY_BENCH_OG(1000000);
```

```
    } catch (pqxx::sql_error const &e) {
        FAIL(std::format("Query failed with: {}", e.what()));
    } catch (std::exception const &e) {
        FAIL(std::format("Failed to connect to PostgreSQL server '{}'",
        ↪  e.what()));
    }
}
```

```
#define JSON_READ_BENCH(N)
↪  \
    BENCHMARK_ADVANCED("JSON Reading " #N)(Catch::Benchmark::Chronometer meter) {
    ↪  \
        std::vector<Salary> salaries;
        ↪  \
        if (auto const error = glz::read_file_json(salaries,
        ↪  "../employees/salary.json", std::string{})) {            \
            FAIL(std::format("Failed to read JSON file: {}",
            ↪  glz::format_error(error)));                          \
        }
        ↪  \
        meter.measure([&] {
        ↪  \
            std::int64_t sum = 0;
            ↪  \
            for (auto const &s: salaries | std::views::take(N))
            ↪  \
                sum += s.amount;
                ↪  \
            return sum;
            ↪  \
        });
        ↪  \
    }

TEST_CASE("retrieval-test-1-json",
↪  "[retrieval-json][retrieval-test-1][retrieval-test]") {
    BENCHMARK_ADVANCED("JSON File Loading")(Catch::Benchmark::Chronometer meter)
    ↪  {
        meter.measure([&] {
            std::vector<Salary> salaries;
            if (auto const error = glz::read_file_json(salaries,
            ↪  "../employees/salary.json", std::string{}))
                FAIL(std::format("Failed to read JSON file: {}",
                ↪  glz::format_error(error)));
            return salaries.size();
        });
    };

    JSON_READ_BENCH(100);
    JSON_READ_BENCH(500);
    JSON_READ_BENCH(1000);
    JSON_READ_BENCH(5000);
    JSON_READ_BENCH(10000);
    JSON_READ_BENCH(50000);
    JSON_READ_BENCH(100000);
    JSON_READ_BENCH(500000);
    JSON_READ_BENCH(1000000);
}
```

```
#define BEVE_READ_BENCH(N)
↪  \
```

```
    BENCHMARK_ADVANCED("BEVE Reading " #N)(Catch::Benchmark::Chronometer meter) {
    ↪ \
        std::vector<Salary> salaries;
        ↪ \
        if (auto const error = glz::read_file_beve(salaries,
        ↪ "../employees/salary.beve", std::string{})) {            \
            FAIL(std::format("Failed to read BEVE file: {}",
            ↪ glz::format_error(error)));                          \
        }
        ↪ \
        meter.measure([&] {
        ↪ \
            std::int64_t sum = 0;
            ↪ \
            for (auto const &s: salaries | std::views::take(N))
            ↪ \
                sum += s.amount;
                ↪ \
            return sum;
            ↪ \
        });
        ↪ \
    }

TEST_CASE("retrieval-test-1-beve",
↪ "[retrieval-beve][retrieval-test-1][retrieval-test]") {
    BENCHMARK_ADVANCED("BEBE File Loading")(Catch::Benchmark::Chronometer meter)
    ↪ {
        meter.measure([&] {
            std::vector<Salary> salaries;
            if (auto const error = glz::read_file_beve(salaries,
            ↪ "../employees/salary.beve", std::string{}))
                FAIL(std::format("Failed to read BEBE file: {}",
                ↪ glz::format_error(error)));
            return salaries.size();
        });
    };

    BEVE_READ_BENCH(100);
    BEVE_READ_BENCH(500);
    BEVE_READ_BENCH(1000);
    BEVE_READ_BENCH(5000);
    BEVE_READ_BENCH(10000);
    BEVE_READ_BENCH(50000);
    BEVE_READ_BENCH(100000);
    BEVE_READ_BENCH(500000);
    BEVE_READ_BENCH(1000000);
}
```